M-Way Tree, M-Way Search Tree & B-Tree

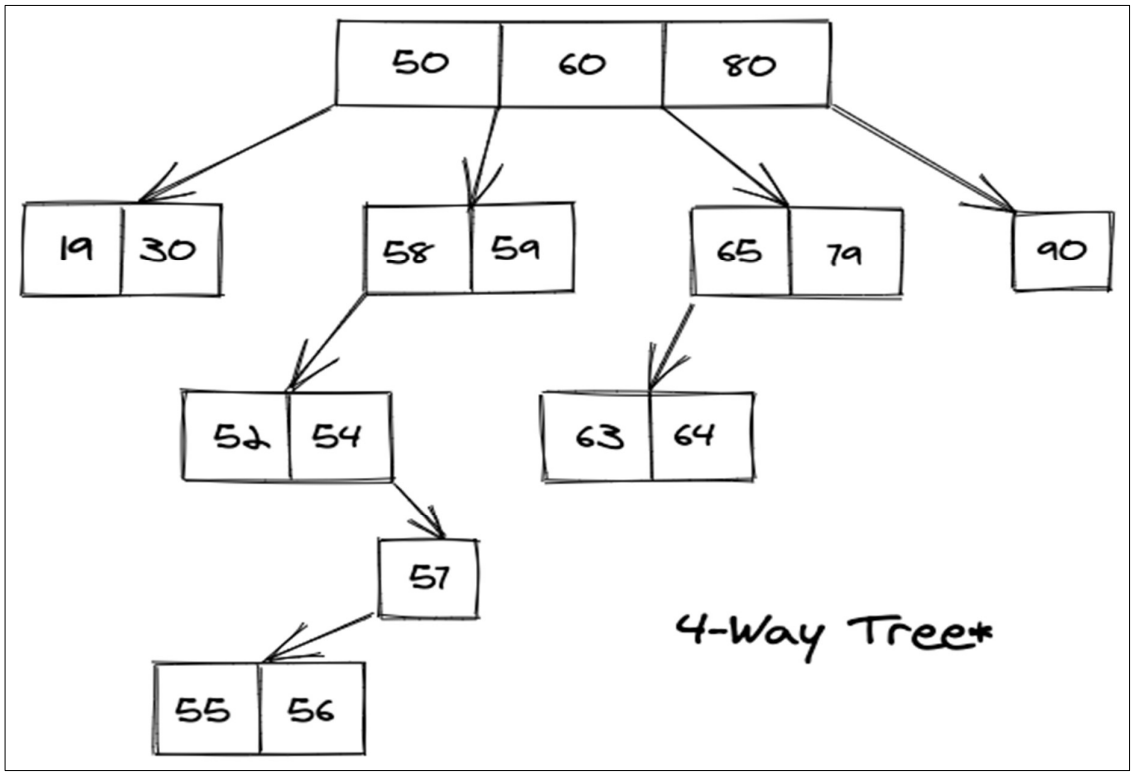
Extended and Threaded Binary Tree

M-way Tree:

An M-way(multi-way) tree is a tree that has the following properties:

- Each node in the tree can have at most **m** children.
- Nodes in the tree have at most **(m-1)** key fields and pointers(references) to the children.

Consider the pictorial representation shown below of an **M-way tree**.



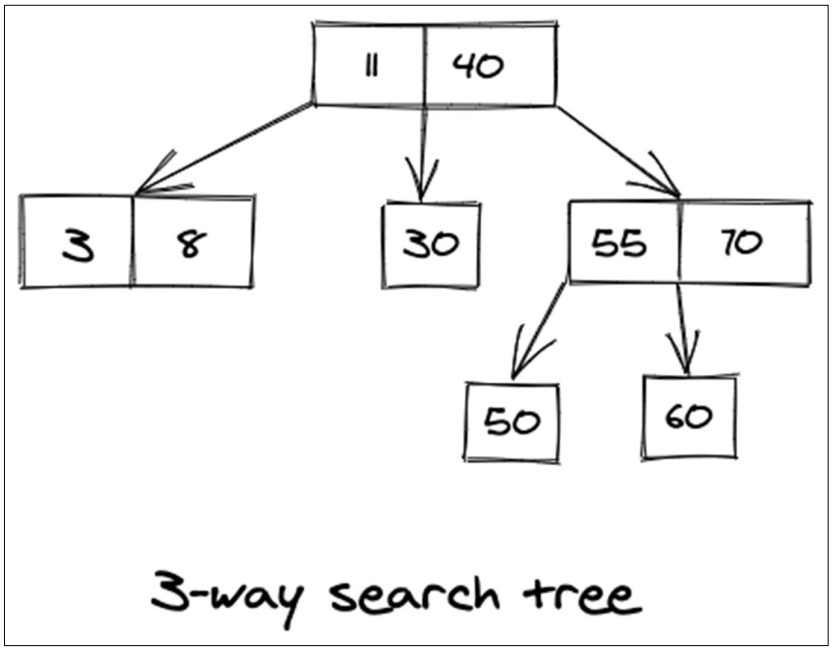
The above image shows a **4-way tree**, where each node can have at most **3(4-1) key fields** and at most **4** children. It is also a **4-way search tree**.

M-way Search Tree:

An M-way search tree is a more constrained m-way tree, and these constrain mainly apply to the key fields and the values in them. The constraints on an **M-way** tree that makes it an M-way search tree are:

- Each node in the tree can associate with **m** children and **m-1** key fields.
- The keys in any node of the tree are arranged in a sorted order(**ascending**).
- The keys in the first **K** children are **less than** the **Kth** key of this node.
- The keys in the last **(m-K)** children are higher than the **Kth** key.

Consider the pictorial representation shown below of an M-way search tree:



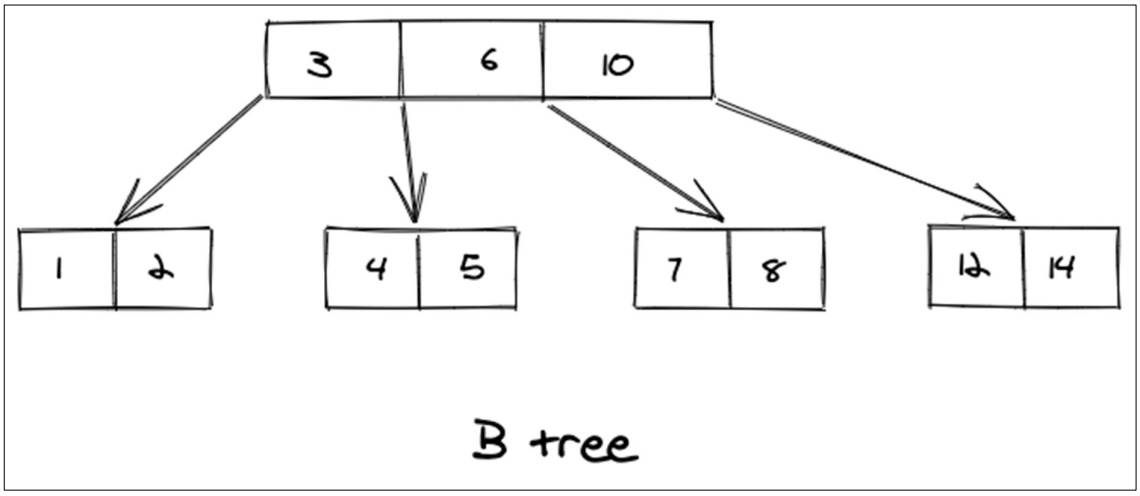
M-way search trees have the same advantage over the M-way trees, which is making the search and update operations much more efficient. Though, they can become unbalanced which in turn leaves us to the same issue of searching for a key in a skewed tree which is not much of an advantage.

B Trees Data Structure:

B tree is an extension of an M-way search tree. Besides having all the properties of an M-way search tree, it has some properties of its own, these mainly are:

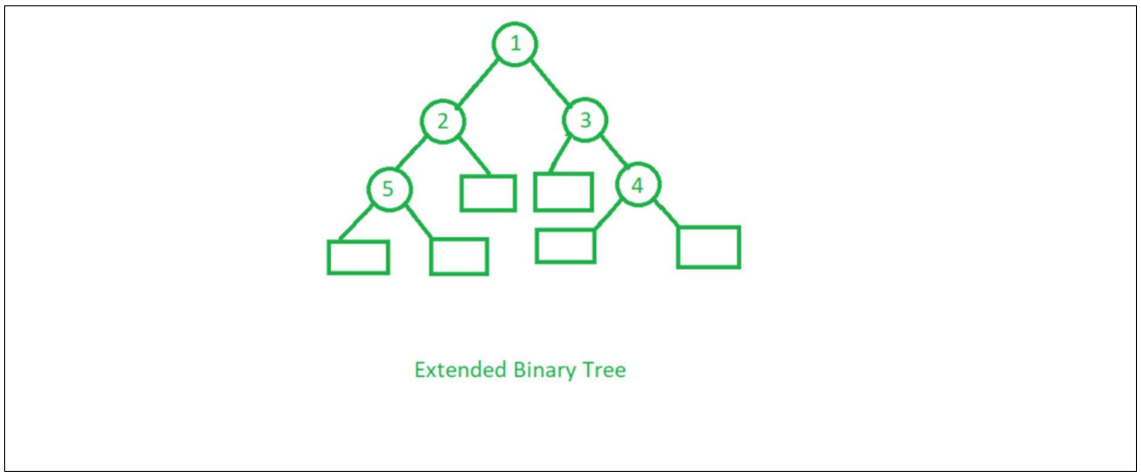
- All the leaf nodes in a B tree are at the same level.
- All internal nodes must have **M/2** children.
- If the root node is a non leaf node, then it must have at least two children.
- All nodes except the root node, must have at least **[M/2]-1** keys and at most **M-1** keys.

Consider the pictorial representation of a B tree shown below:



Extended Binary Tree:

Extended binary tree is a type of binary tree in which all the null sub tree of the original tree are replaced with special nodes called **external nodes** whereas other nodes are called **internal nodes**



Extended Binary Tree

Here the circles represent the internal nodes and the boxes represent the external nodes.

Properties of External binary tree

1. The nodes from the original tree are internal nodes and the special nodes are external nodes.
2. All external nodes are leaf nodes and the internal nodes are non-leaf nodes.
3. Every internal node has exactly two children and every external node is a leaf. It displays the result which is a complete binary tree.

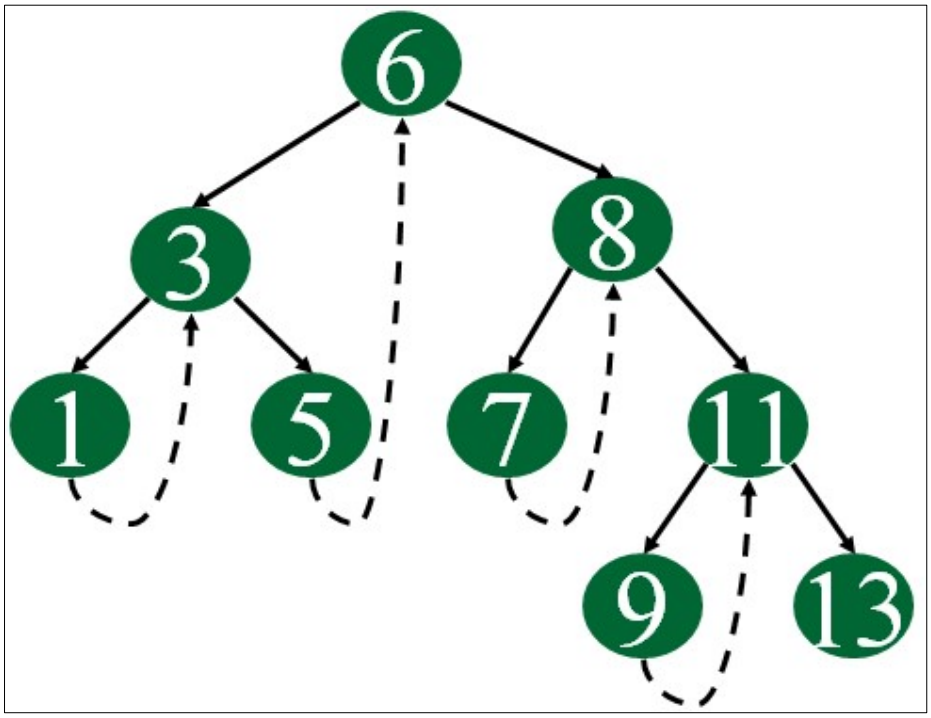
Applications:

- Implementing the network Routing Tables
- Calculating the weighted Path
- Converting Binary Tree to Complete Binary Tree, etc.

Threaded Binary Tree:

A threaded binary tree is a type of binary tree data structure where the empty left and right child pointers in a binary tree are replaced with threads that link nodes directly to their in-order predecessor or successor, thereby providing a way to traverse the tree without using recursion or a stack.

Threaded binary trees can be useful when space is a concern, as they can eliminate the need for a stack during traversal. However, they can be more complex to implement than standard binary trees.



Advantages of Threaded Binary Tree

- In this Tree it enables linear traversal of elements.
- It eliminates the use of stack as it perform linear traversal, so save memory.
- Enables to find parent node without explicit use of parent pointer
- Threaded tree give forward and backward traversal of nodes by in-order fashion
- Nodes contain pointers to in-order predecessor and successor

- For a given node, we can easily find inorder predecessor and successor. So, searching is much more easier.
- In threaded binary tree there is no NULL pointer present. Hence memory wastage in occupying NULL links is avoided.
- The threads are pointing to successor and predecessor nodes. This makes us to obtain predecessor and successor node of any node quickly.
- There is no need of stack while traversing the tree, because using thread links we can reach to previously visited nodes.

Disadvantages of Threaded Binary Tree

- Every node in threaded binary tree need extra information(extra memory) to indicate whether its left or right node indicated its child nodes or its inorder predecessor or successor. So, the node consumes extra memory to implement.
- Insertion and deletion are way more complex and time consuming than the normal one since both threads and ordinary links need to be maintained.
- Implementing threads for every possible node is complicated.
- **Increased complexity**: Implementing a threaded binary tree requires more complex algorithms and data structures than a regular binary tree. This can make the code harder to read and debug.
- **Extra memory usage**: In some cases, the additional pointers used to thread the tree can use up more memory than a regular binary tree. This is especially true if the tree is not fully balanced, as threading a skewed tree can result in a large number of additional pointers.
- **Limited flexibility**: Threaded binary trees are specialized data structures that are optimized for specific types of traversal. While they can be more efficient than regular binary trees for these types of operations, they may not be as useful in other scenarios. For example, they cannot be easily modified (e.g. inserting or deleting nodes) without breaking the threading.
- **Difficulty in parallelizing**: It can be challenging to parallelize operations on a threaded binary tree, as the threading can introduce data dependencies that make it difficult to process nodes independently. This can limit the performance gains that can be achieved through parallelism.

Applications of threaded binary tree –

- **Expression evaluation**: Threaded binary trees can be used to evaluate arithmetic expressions in a way that avoids recursion or a stack. The tree can be constructed from the input expression, and then traversed in-order or pre-order to perform the evaluation.
- **Database indexing**: In a database, threaded binary trees can be used to index data based on a specific field (e.g. last name). The tree can be constructed with the indexed values as keys, and then traversed in-order to retrieve the data in sorted order.
- **Symbol table management**: In a compiler or interpreter, threaded binary trees can be used to store and manage symbol tables for variables and functions. The tree can be constructed with the symbols as keys, and then traversed in-order or pre-order to perform various operations on the symbol table.
- **Disk-based data structures**: Threaded binary trees can be used in disk-based data structures (e.g. B-trees) to improve performance. By threading the tree, it can be traversed in a way that minimizes disk seeks and improves locality of reference.
- **Navigation of hierarchical data**: In certain applications, threaded binary trees can be used to navigate hierarchical data structures, such as file systems or web site directories. The tree can be constructed from the hierarchical data, and then traversed in-order or pre-order to efficiently access the data in a specific order.